Lists, Dictionaries & Tuples

Bit by Bit Coding - Term 2 | Week 1

Key Concepts

- list: ordered, changeable collection, uses [ ].
- dict: key-value pairs, uses { }, access by key.
- tuple: ordered, UNchangeable collection, uses ( ).
- Indexing starts at 0; negative indices count from the end.
- Slicing returns a sub-sequence: data[start:stop].

Syntax Reference

Lists

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0])        # apple
print(fruits[-1])       # cherry (last)
print(fruits[1:3])      # ['banana','cherry']
fruits.append("date")   # add to end
fruits.remove("apple")  # remove by value
print(len(fruits))      # length
```

List Methods

- .append(x) add to end - .insert(i, x) insert at index.
- .remove(x) remove first match - .pop() remove & return last.
- .sort() sort in place - .reverse() reverse in place.

Dictionaries

```
student = {"name": "Ali", "age": 15, "grade": "B"}
print(student["name"])        # Ali
student["age"] = 16           # update
student["school"] = "BbB"    # add a key
for key in student:
    print(key, student[key])
```

Dict Methods

- .keys() all keys - .values() all values - .items() pairs.
- .get(key, default) safe access that won't crash if missing.

Tuples

```
point = (3, 4)
x, y = point        # unpacking
print(point[0])     # 3
# point[0] = 5      # ERROR: tuples can't change
```

List Comprehensions

```
squares = [n*n for n in range(5)]   # [0,1,4,9,16]
evens = [n for n in nums if n % 2 == 0]
```

Common Patterns

- Iterate pairs: for k, v in d.items():.
- Count occurrences: counts[w] = counts.get(w, 0) + 1.

- Build a list from a loop, or use a comprehension for brevity.

Tips & Common Mistakes

- Lists use [], dicts use {}, tuples use ( ).
- Dict keys must be unique - repeating a key overwrites it.
- Index out of range: fruits[10] on a short list raises an error.
- Tuples are immutable - you can't .append() or reassign items.